

Python Project Structure: How to Choose the Right Layout

Cheat Sheet: Flat vs. `src/` Layout

A companion to the **Real Python tutorial** of the same name. Visit **realpython.com** to read the full guide.

Vocabulary first. The **project** is the whole repository (config, tests, docs); the **package** is the importable code inside it. Almost every layout question comes down to **where the package sits** relative to everything else.

STEP 1 Name the project type

Type	Layout	Why
Script	none (flat if it grows)	No install needed — at most a PEP 723 header.
Application	flat	Frameworks ship flat; go `src/` when packaging matters.
Library	src/	Install is part of the user contract — you ship to PyPI.

When to reach for each

flat

- Script or small CLI
- A framework expects it
- Already-flat repo
- You want the simplest setup

src/

- Publishing to PyPI
- CI must catch packaging bugs
- Grown past ~10 modules
- Wheel must match the repo

Rule of thumb: start flat; switch to `src/` when install correctness starts to matter. Migration is one config line:

`pyproject.toml`

```
[tool.setuptools.packages.find]
where = ["src"]
```

STEP 2 Pick a layout: flat vs. `src/`

Only difference: **how deep the importable package sits**.

`flat — package at the project root`

```
project_name/
├── project_name/
│   └── __init__.py
├── tests/
├── pyproject.toml
└── README.md
```

`src/ — package one level deeper`

```
project_name/
├── src/
│   └── project_name/
│       └── __init__.py
├── tests/
├── pyproject.toml
└── README.md
```

flat Import and run **without installing**.

src/ Nothing imports until `python -m pip install -e .` — tests exercise the **same package your users get**.

STEP 3 Organize inside-out

- **Modules** — the `.py` files. `__init__.py` marks the package and re-exports its public API via `__all__`.
- **Entry point** — a thin `__main__.py` makes it runnable with `python -m package_name`.
- **Subpackages** — domain folders (`core/`, `api/`, `cli/`) beat a catch-all `utils.py`. Default to **absolute imports**.

`pyproject.toml — turn the package into a command`

```
[project.scripts]
package_name = "package_name.cli:main"
```

On install, this puts a `package_name` command on PATH, wired to `main()` — no `python -m` needed.

Gotchas

- **Flat hides packaging bugs.** A module missing from the wheel still imports from your working dir — run tests against an **installed copy in CI**.
- **`py.typed` (PEP 561).** Add the empty marker inside the package so type checkers read your hints; without it, every export is `Any`.
- **Tests live outside the package** in a top-level `tests/`, so they stay out of the wheel.

Real layouts at a glance

Project	Layout	Entry point	Code split by	Ships as
Library	src/	none (import only)	Modules	PyPI wheel
CLI	flat	`__main__.py` + `[project.scripts]`	Modules + a subpackage	CLI binary
Django	flat	`manage.py` + WSGI app	Django apps	Container / host
Flask	flat	Application factory	Blueprints	Container / host
FastAPI	flat	ASGI app in `main.py`	`APIRouter` subpackages	Container / host
GUI	flat	`__main__.py` + `[project.scripts]`	MVC / MVP / MVVM layers	Frozen executable

Flat dominates applications; the library reaches for `src/`. Every runnable project funnels through one thin entry point — the framework decides how you furnish the skeleton.